NVIDIA

Sparser MoE Explorations

Router, communication, and systems co-design at 2304 experts

Harry Zhou | Mentor: Robin Zhang

Internship final presentation | July 2026

Contents

Introduction

Sparser MoE Trends

Sparser routing is one possible scaling direction, not a universal design

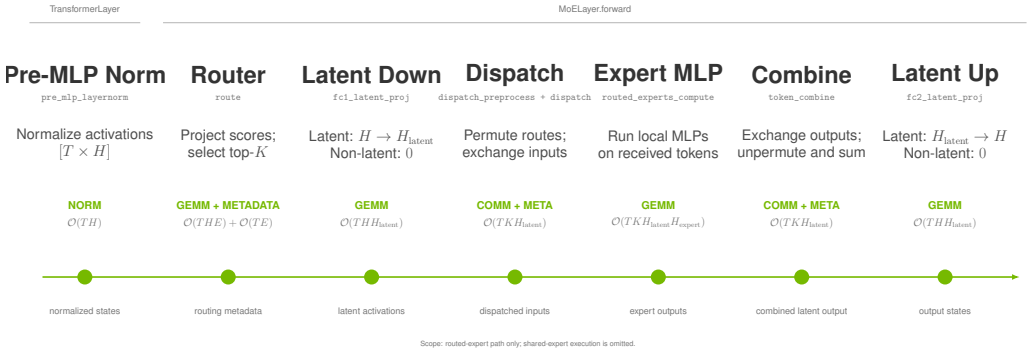
Open model	Scale	Routed experts	Selected/token
Mixtral 8×7B	Early open sparse MoE	8	2
DeepSeek-V3	671B total / 37B active	256	8
Nemotron 3 Ultra	550B total / 55B active	512	22
DeepSeek-V4-Pro	1.6T total / 49B active	384	6
Kimi K2	1T total / 32B active	384	8
Kimi K3	2.8T total / 104B active	896	16

Trend

LatentMoE reduces the routed representation before expert computation. It provides one path to increase routed-expert count or top-k while controlling the sparse-compute budget.

MoE Execution

MCore carries activations and routing metadata through the sparse MLP



Optimization target. Sparse performance depends on router work, metadata transformation, token exchange, expert kernels, and weighted recombination.

LatentMoE Cost Model

Latent representations reduce exchange payload, not expert computation

Expert computation

$$\text{FLOPs}_{\text{expert}} \propto TKH_{\text{latent}}H_{\text{expert}}$$

Every selected route still executes its expert MLP. On a non-latent path, down/up projections are absent (zero cost) and $H_{\text{latent}} = H$.

Token exchange

$$\text{payload} \propto TKH_{\text{latent}}$$

Router score projection costs $\mathcal{O}(THE)$ and top-k selection adds $\mathcal{O}(TE)$ metadata work. Latent MoE narrows the exchanged payload when $H_{\text{latent}} \ll H$; without it, $H_{\text{latent}} = H$.

Sparse-path budget. Selection, metadata transformation, communication, expert compute, and combine remain coupled.

Target Workload

Contents

Target Workload

Experimental Configuration

A Qwen3-Next-derived latent-MoE training model isolates large-expert sparse-path costs

Backbone & Attention

24 layers; $H = 2048$
GQA: 32 Q / 2 KV groups; GDN disabled

Latent MoE

2304 experts; top-$k = 36$; $H_{\text{latent}} = 512$
FFN/shared width 512; EP72; 32 E/rank

4.5×
router scores
512 → 2304

3.6×
selected routes
10 → 36

Recipe

4K tokens; GB200 NVL72 and GB300 NVL72.
TP1 / PP1 / EP72; MBS 3 / GBS 864; MXFP8.
1F1B; paged stash; full-iteration CUDA graph.

Initial Analysis

Contents

Initial Analysis

Initial Profile

Rank-0 aggregate GPU time identifies the initial bottlenecks

Kernel group	GPU time
Fused router	3947.5 ms
HybridEP combine	1713.9 ms
HybridEP dispatch	1259.5 ms
FlashAttention	903.9 ms
Other GEMM	371.1 ms
Grouped GEMM (expert MLP)	286.1 ms

Interpretation

The fused router dominates grouped expert-MLP compute by 13.8×. HybridEP combine and dispatch together contribute 10.4× the grouped-GEMM time.

These sums cover all captured forward and backward launches. Stream overlap means they are not additive wall-clock time.

First remove launch gaps; then optimize the router and the HybridEP path.

Launch-Overhead Mitigation

Upstream runtime work establishes the replayable baseline

Most of the launch-overhead reduction comes from upstream execution-system work:

- CuTe DSL sync-free grouped GEMM and fused GroupedMLP reduce expert-MLP launches and synchronizations.
- Paged stash makes activation storage capture-compatible.
- Full-iteration CUDA graphs remove CPU launch gaps and stabilize replay.
- Fused quantization removes remaining small-kernel work; larger microbatches amortize the residual.

This work targets the router and HybridEP bottlenecks on top of that baseline.

Router Optimization

Contents

Fused-Router Optimization

Radix Selection

TE PR #2821 replaces repeated maximum selection with a one-warp radix selector

Repeated maximum selection

A rank-r pass scans E scores and checks up to $r-1$ prior IDs:

$$\sum_{r=1}^{K} E\, r = O(EK^2).$$

At $E = 2304$, $K = 36$, selection repeats a full expert-row walk for every selected rank.

Radix selection

FP32 order bits are narrowed by eight 4-bit passes, followed by a constant number of gathers:

$$8E + 2E = O(E).$$

At $E = 2304$, per-thread selection needs a 9 KB score row per lane. CTA-wide selection adds inter-warp barriers and shared-memory coordination. TE retains one warp per token: lanes stride across experts and use shuffle collectives.

2304
experts
one score row per token

36
selected IDs
large sparse fan-out

8
radix passes
FP32, 4 bits/pass

Radix Selection: Algorithm

Each pass narrows the rank-K score prefix without materializing a sort

```
ordered = float_to_ordered_uint(scores)
prefix, mask = 0, 0
rank = topk

for pass in range(7, -1, -1):
    hist[0:16] = count scores matching prefix
    bucket = highest_bucket_containing(rank)
    prefix |= bucket << (4 * pass)
    mask   |= 0xf << (4 * pass)

gather scores > prefix
gather ties in index order until topk
```

Two phases

1. Count matching scores in 16 buckets; choose the bucket containing rank K.
2. Gather scores above the final value; use ordered ties to complete exactly K IDs.

Invariant

The selected set is exact. Radix changes how the threshold is found, not the top-k semantics or deterministic tie handling.

Radix Selection: Implementation

The CUDA kernel carries the rank state in registers and uses warp collectives

```
constexpr int kPasses = 8;
unsigned prefix = 0, mask = 0;
for (int pass = kPasses - 1; pass >= 0; --pass) {
  unsigned packed[4] = {};
  count_matching_scores(packed, prefix, mask);
  int bucket = find_target(packed, k_remaining);
  prefix |= unsigned(bucket) << (4 * pass);
  mask   |= 0xfu << (4 * pass);
}
```

One warp / token

The implementation maps a token row to a warp. Each lane strides across experts, then warp reductions combine 16 bucket counts.

Bounded support

Packed 8-bit counters support $E \leq 8160$; the 2304-expert target is well inside that range.

Secondary Optimizations

PR #3012 turns the radix algorithm into a throughput-oriented kernel family

Loop fusion	Async load	Packed radix	Static score	Merged head
Merge clear, load, score, save, and bias work where the score path permits.	Double-buffer raw logits and use a persistent grid sized by occupancy.	Store sixteen counters in four 32-bit registers instead of 32 registers.	Instantiate score functions at compile time to remove hot-path branches.	Keep a specialized small-k head while routing large shapes to the optimized head.

Design principle

The algorithmic fix removed the large-K cliff. These refinements remove data movement, register, branch, and occupancy costs exposed after that cliff is gone.

Secondary Optimizations: Loop Fusion

The score path writes each expert value once instead of staging separate loops

```
if constexpr (ScoreFunc == kSigmoid) {
  for (int i = lane_id; i < E; i += 32) {
    float value = sigmoid(raw_logits[i]);
    intermediate[pos + i] = value;
    if (bias) value += bias[i];
    scores[i] = value;
  }
} else if constexpr (ScoreFunc == kSqrtsoftplus) {
  for (int i = lane_id; i < E; i += 32) {
    intermediate[pos + i] = raw_logits[i];
    scores[i] = sqrtsoftplus(raw_logits[i]);
  }
}
```

Fused dataflow

A single strided loop converts logits, evaluates the score function, preserves the value required by backward, applies optional bias, and populates selection input. The softmax path

remains multi-pass because its normalization is mathematically coupled across the entire expert row.

Secondary Optimizations: Async Loading

Raw logits are prefetched with `cp.async` while the current tile is selected

```
// tile n+1 starts before tile n selects
loader.start_load(next_logits, E, lane_id);

loader.wait();
select_topk(loader.current_buf());

// 16-byte global-to-shared copy
cp_async_16B(dst, src);
cp_async_commit();
```

Overlap policy

The loader keeps the original input dtype in shared memory, deferring conversion to compute. It uses 16-byte vector copies on supported GPUs and a scalar fallback otherwise. The host launcher

chooses one or two buffers from the shared-memory budget, then sizes the persistent grid from active blocks per SM.

Secondary Optimizations: Packed Radix

Eight-bit fields lower histogram bookkeeping from 32 registers to four

```
// 16 counters in 4 packed u32 values
unsigned packed[4] = {};
int bucket = (u >> digit_pos) & 0xf;
int pack = bucket >> 2;
int shift = (bucket & 3) << 3;
packed[pack] += 1u << shift;

unsigned count = (packed[b >> 2] >> ((b & 3) << 3)) & 0xffu;
unsigned total = warp_allreduce_sum(count);
```

Why it is safe

Each lane sees at most $\lceil E/32 \rceil$ scores per bucket. For $E = 2304$, the largest count is 72, which fits in an 8-bit field. The implementation

enforces $E \leq 8160$ so a packed counter cannot overflow.

Secondary Optimizations: Static Scores

Compile-time score specialization removes the runtime score-function branch

```
template <typename T, TopkFuncType TopkFunc, int ScoreFunc>
__global__ void router_forward(...);

if constexpr (ScoreFunc == kSoftmax)
  softmax(scores, E, lane_id);
else if constexpr (ScoreFunc == kSigmoid)
  scores[i] = sigmoid(raw_logits[i]);
else if constexpr (ScoreFunc == kSqrtsoftplus)
  scores[i] = sqrtsoftplus(raw_logits[i]);
}
```

Specialization

The launcher performs the score-function switch once, then each instantiated kernel contains only its required math and synchronization. This benefits both top-k and

auxiliary-loss forward/backward paths, where branch structure otherwise repeats for every expert.

Secondary Optimizations: Merged Head

A single launcher preserves a lightweight head for small k and the optimized head for large shapes

```
bool use_radix = topk >= radix_threshold && E <= kMaxExperts;

if (!use_radix)
  launch_simple(simple_kernel<...>);
else
  launch(optimized_kernel<..., Radix, ScoreFunc>);
```

Shape-aware dispatch

The simple head avoids async-loader and persistent-grid overhead on conventional small-k shapes. The optimized head is selected only when radix work dominates. This is why gains at

$2304/36$ do not require regressing common router shapes.

Dense Routing Map

TE PR #3129 replaces the $[T, E]$ byte map with selected int16 IDs

Per-token representation

Format	Shape	Bytes/token
Byte map	$[E]$	2304
Selected IDs	int16 $[K]$	72

$$\frac{E}{2K} = \frac{2304}{2 \times 36} = 32 \times .$$

Target batch

Local metadata, $T = 8192$	Size
Byte $[T, E]$	18.9 MB
int16 $[T, K]$	0.59 MB

The compact buffer records expert IDs only.

Probabilities remain separate because training needs selected weights and their backward dependencies.

Dense Routing Map: Forward

The index-output API emits one int16 ID per selected route

```
for (int i = lane_id; i < topk; i += 32) {
  int expert = topk_indices[i];
  if (indices_out)
    indices_out[token * topk + i] = static_cast<IndexType>(
      expert);
  probs[pos + expert] = scale * topk_scores[i];
}

if constexpr (IndexType == int16_t)
  NVTE_CHECK(E <= INT16_MAX, ...);
```

Producer contract

The selected ID output is optional: legacy bytemap/bitmap callers retain their existing format, while dense-route callers receive $[T, K]$ indices directly from the fused selection result. $E = 2304$ is safely representable in int16.

Dense Routing Map: Backward

The dedicated index path traverses selected IDs and zero-fills the dense gradient

```
const IndexType* ids = topk_indices + token * topk;
zero(grad_logits + pos, E);

for (int k = lane_id; k < topk; k += 32) {
  int expert = static_cast<int>(ids[k]);
  auto grad = grad_probs[pos + expert] * scale;
  grad_logits[pos + expert] = grad;
}
```

Complexity change

Replace repeated membership tests of every expert against a K-entry list with a dense zero-fill plus direct writes to selected locations:

$$O\big(T(E+K)\big) \text{ instead of } O\big(TEK\big).$$

Full-row score-function coupling still performs the mathematically required row work; the redundant membership scan is removed.

Router SOL

Parametric roofs for FP32 top-k forward

HBM traffic roof

B200 and B300 raw HBM rooflines are both 8 TB/s. The benchmark's minimum global traffic is

$$B_{\text{sparse}} = 13\,TE, \qquad B_{\text{int16}} = 12\,TE + 2\,TK.$$

The terms are logits read, probability and FP32-intermediate writes, plus either a dense bool map or K int16 indices. Thus

$$t_{\text{HBM}} = B_{\text{eff}}/(8\ \text{TB/s}).$$

This is an effective-byte roof, not a measurement of physical HBM traffic.

Compute roof

R_{issue} is the sustained CUDA-core issue rate. Full forward work is

$$W = TEI_{\text{row}} + (8+G)\,TEI_{\text{radix}} + TKI_{\text{selected}}, \qquad 1 \le G \le 2.$$

I_{row}: score evaluation, initialization, full-row stores; I_{selected}: normalization and selected writes.

$$t_{\text{comp}} = W/R_{\text{issue}}, \qquad BW_{\text{comp,eq}} = B_{\text{eff}}/t_{\text{comp}}.$$

FP32 TFLOP/s does not directly give R_{issue} for this integer/collective path; dependencies further lower the achievable rate.

SOL Examples

Radix work and effective traffic scale together

Int16 dense output gives $B_{\text{eff}}/T = 12E + 2K$ bytes. Radix selection scans $(8 + G)E$ scores/token; the table gives $(B_{\text{eff}}/T) \,/\, ((8 + G)E)$.

Shape E/K	B_{eff}/T	Radix scans$/T$	Conversion factor
$512/16$	6.18 KB	4608–5120	1.21–1.34 B/scan
$896/16$	10.78 KB	8064–8960	1.20–1.34 B/scan
$2304/36$	27.72 KB	20736–23040	1.20–1.34 B/scan

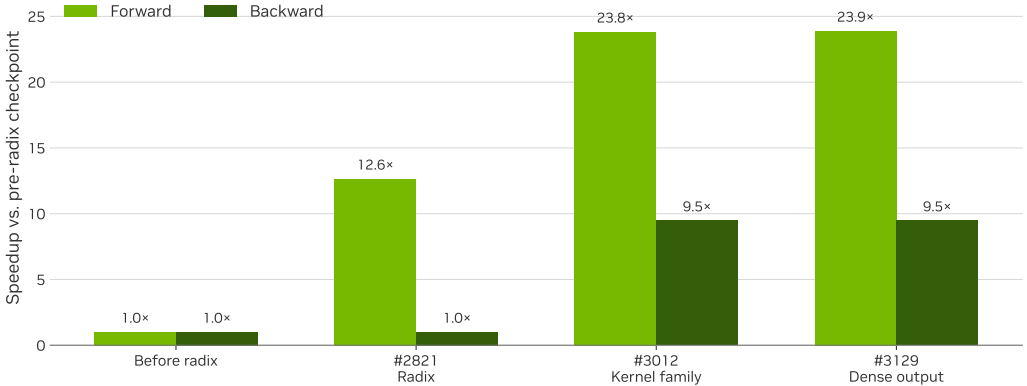
If hypothetical $R_{\text{issue}} = 75$ Tinst/s and $I_{\text{radix}} = 1$ are used, the radix-only conversion is 90–100 TB/s. Full-kernel score, normalization, and store work lower the compute roof; this is a normalization, not a one-operation SOL.

Observed forward bandwidth.

B300, FP32 softmax, dense int16, $T = 65536$, PR #3129: $896/16 = 1000.5$ GB/s; $2304/36 = 956.8$ GB/s. Both are effective bandwidths, well below the 8 TB/s HBM roof.

Router Performance

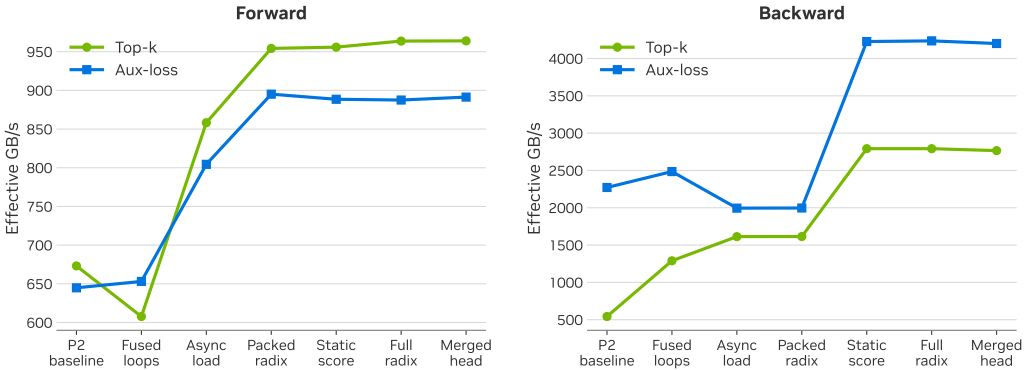
Matched B300 trace-back: 65536 tokens, 2304 experts, top-$k = 36$, softmax



Radix selection removes the large-K penalty; the following kernel and metadata changes retain that gain through the final dense-routing implementation.

Router Performance

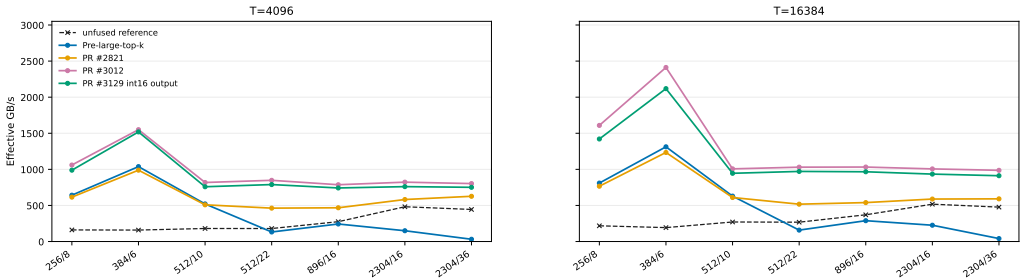
Incremental B300 result: 8192 tokens, 2304 experts, top-$k = 36$, softmax



Loop fusion, asynchronous loading, packed radix state, static-score specialization, and the merged head provide cumulative improvements across the top-K and auxiliary-loss paths.

Router Performance

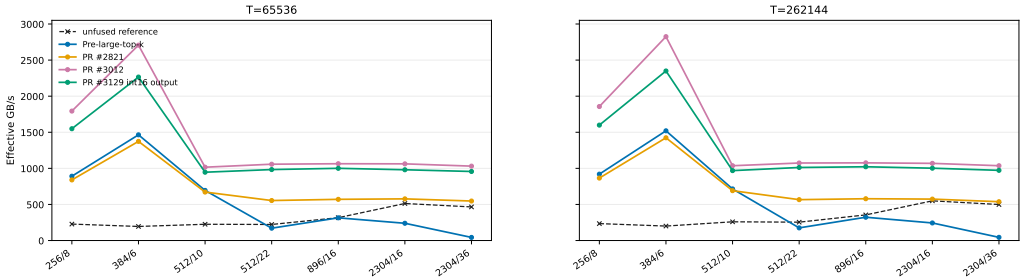
Single B300 router benchmark: softmax; seven shapes; $T = 4\text{K}$ and 16K



The final dense-int16 output checkpoint sustains the forward gain across the measured shapes; the first two token counts expose the launch-amortization regime.

Router Performance

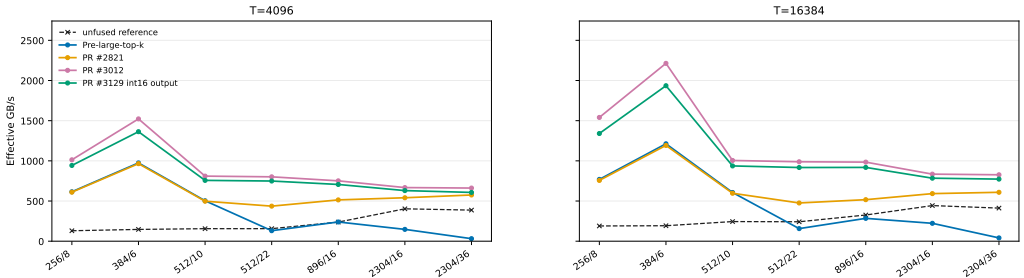
Single B300 router benchmark: softmax; seven shapes; $T = 64$K and 256K



At steady-state token counts, the same checkpoint retains the gain while effective bandwidth approaches its persistent-kernel operating range.

Router Performance

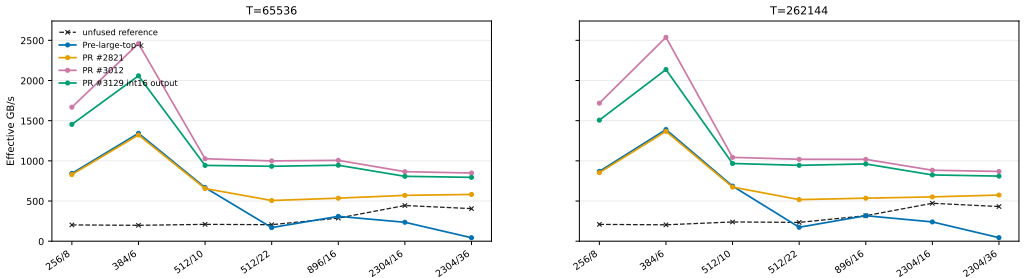
Single B300 router benchmark: sigmoid; seven shapes; $T = 4\text{K}$ and 16K



The same selection and metadata improvements transfer to sigmoid scoring; its score arithmetic changes the absolute bandwidth level relative to softmax.

Router Performance

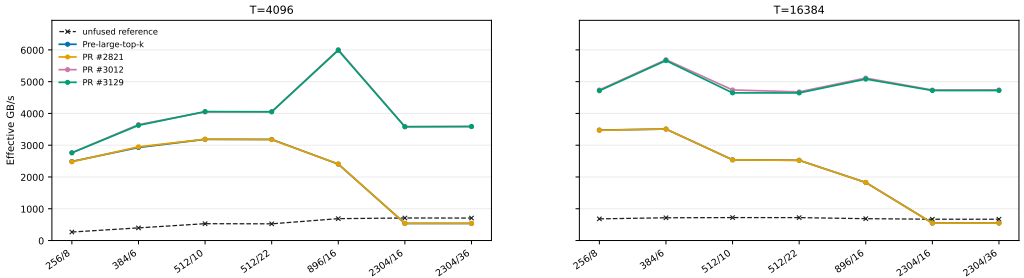
Single B300 router benchmark: sigmoid; seven shapes; $T = 64$K and 256K



The sigmoid forward result follows the same trend at steady state, with its score function setting the attainable effective bandwidth.

Router Performance

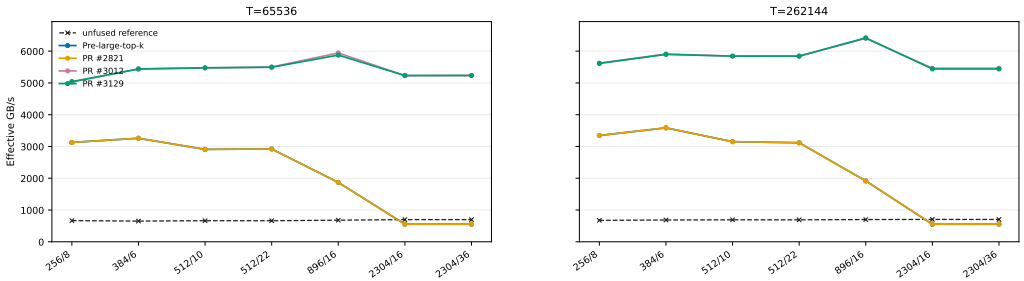
Single B300 router benchmark: softmax backward; seven shapes; $T = 4\text{K}$ and 16K



Selected-index routing eliminates redundant membership work in top-K backward, with the benefit most visible at large expert counts.

Router Performance

Single B300 router benchmark: softmax backward; seven shapes; $T = 64$K and 256K

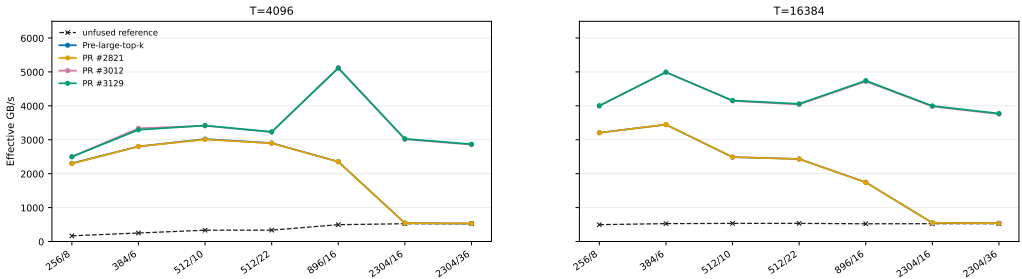


At steady state, selected-index traversal keeps the full-row routing-map scan out of the backward path.

NVIDIA

Router Performance

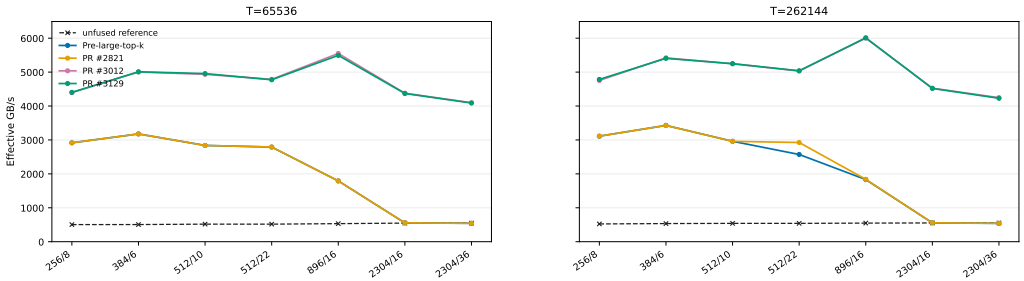
Single B300 router benchmark: sigmoid backward; seven shapes; $T = 4\text{K}$ and 16K



Backward gains persist for sigmoid because selected-index traversal removes the same full-row routing-map scan, independent of the score function.

Router Performance

Single B300 router benchmark: sigmoid backward; seven shapes; $T = 64\text{K}$ and 256K

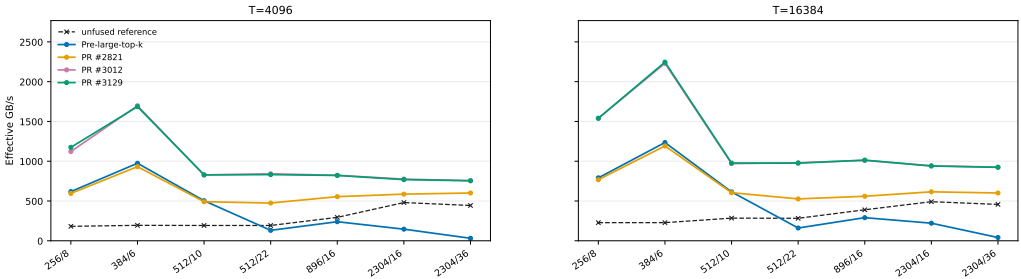


The steady-state sigmoid backward sweep retains the selected-index benefit across all measured shapes.

NVIDIA

Router Performance

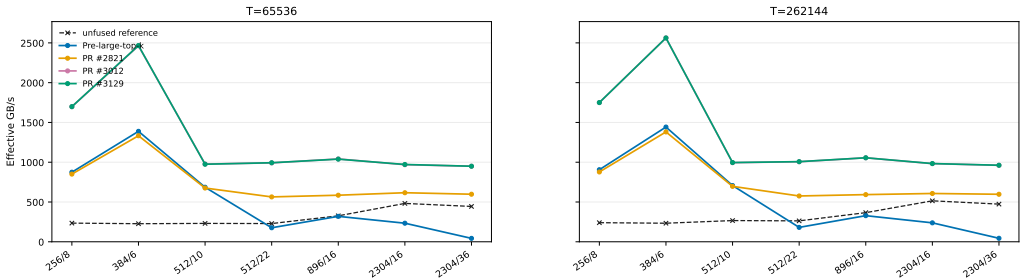
Single B300 router benchmark: softmax auxiliary loss; $T = 4\text{K}$ and 16K



Auxiliary-loss forward shares the optimized loading and score-processing structure across the seven shape pairs.

Router Performance

Single B300 router benchmark: softmax auxiliary loss; $T = 64$K and 256K

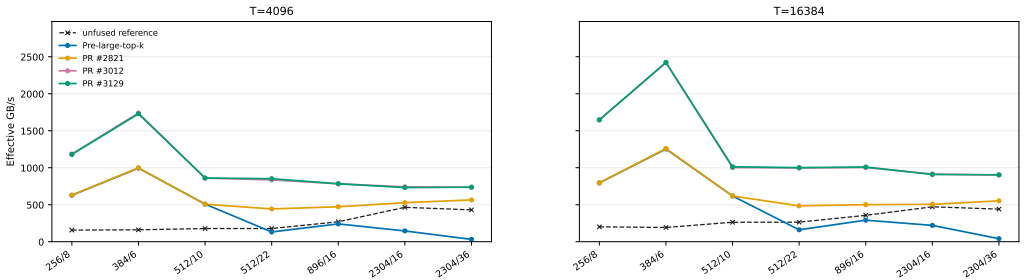


The steady-state sweep retains the auxiliary-loss forward gain as the persistent kernel gains sufficient work.

NVIDIA

Router Performance

Single B300 router benchmark: sigmoid auxiliary loss; $T = 4\text{K}$ and 16K

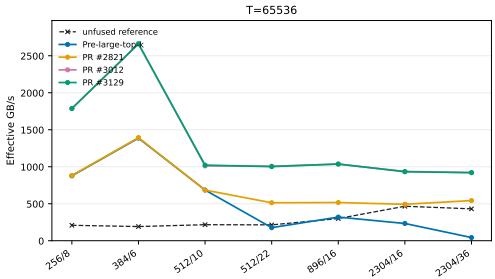


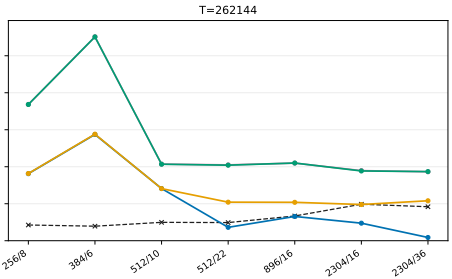
Sigmoid auxiliary-loss forward follows the same direction; score arithmetic sets the absolute bandwidth level.

NVIDIA

Router Performance

Single B300 router benchmark: sigmoid auxiliary loss; $T = 64$K and 256K

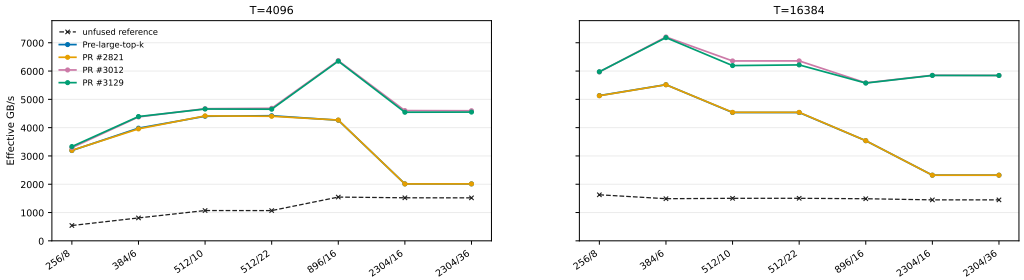




At steady state, the optimized score path sustains the gain across the complete shape sweep.

Router Performance

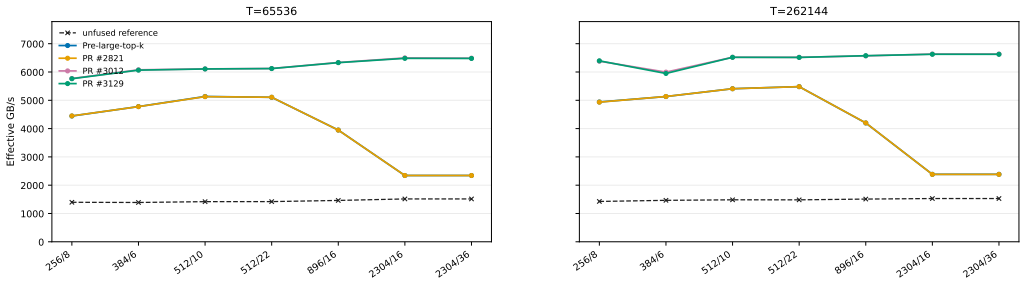
Single B300 router benchmark: softmax auxiliary backward; $T = 4\text{K}$ and 16K



The backward kernel benefits from optimized per-row reductions and reduced shared-memory pressure.

Router Performance

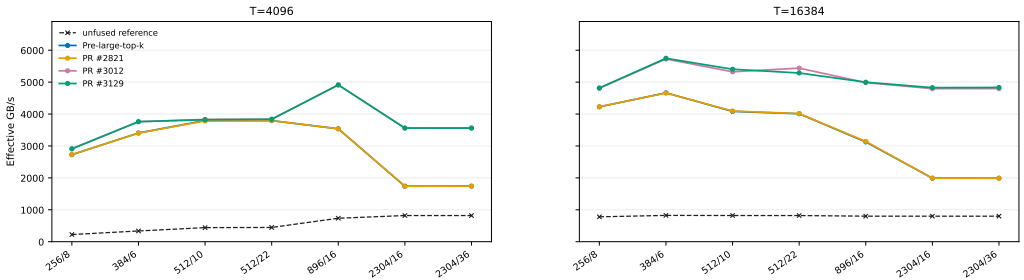
Single B300 router benchmark: softmax auxiliary backward; $T = 64$K and 256K



The steady-state sweep retains the large-shape benefit once the persistent kernel has enough work.

Router Performance

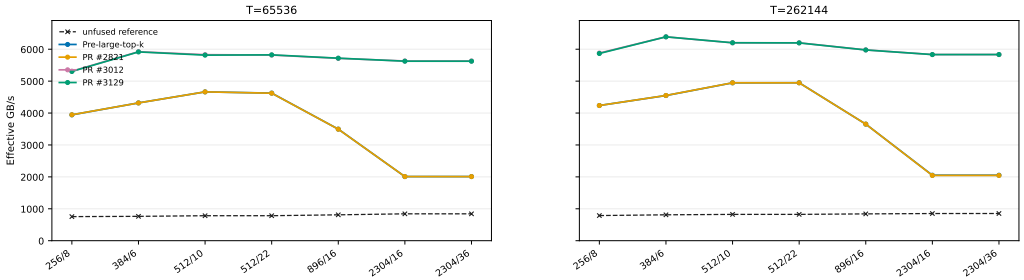
Single B300 router benchmark: sigmoid auxiliary backward; $T = 4\text{K}$ and 16K



The sigmoid auxiliary backward sweep retains complete score-function semantics at low token counts.

Router Performance

Single B300 router benchmark: sigmoid auxiliary backward; $T = 64$K and 256K



The steady-state results show the same large-shape benefit while retaining the complete score function.

NVIDIA

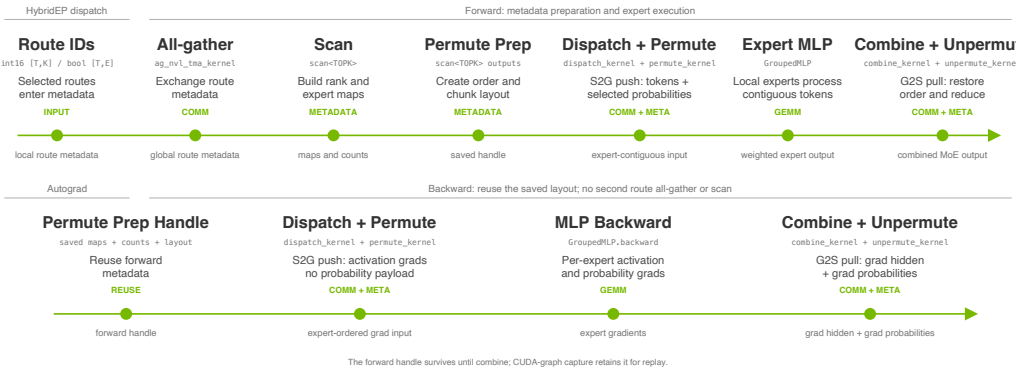
HybridEP Optimization

Contents

HybridEP Optimization

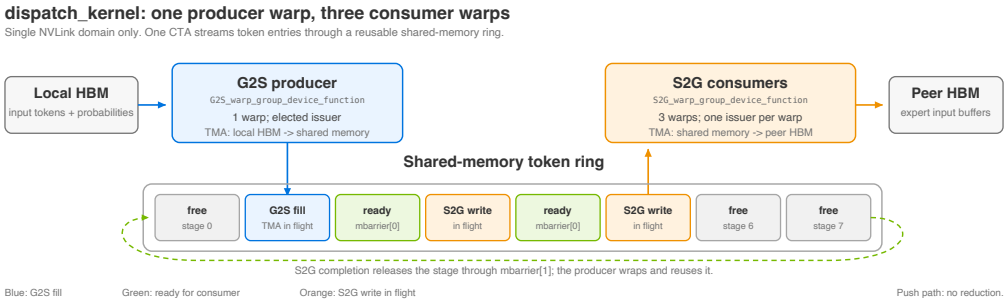
HybridEP Pipeline

Metadata is prepared once; the saved handle drives both directions



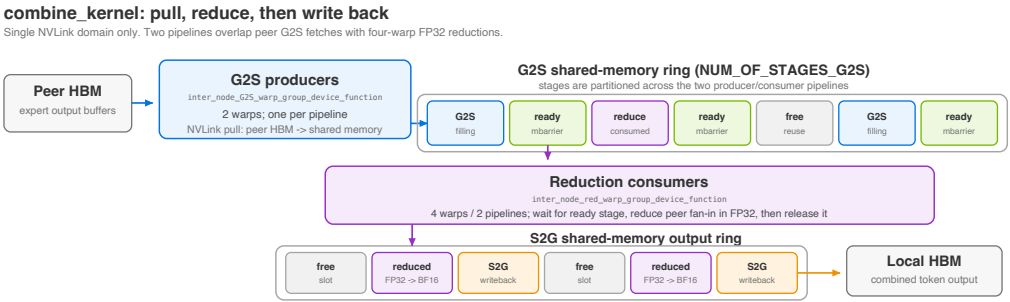
Dispatch Pipeline

Single NVLink domain: G2S producer and S2G consumer warps



Combine Pipeline

Single NVLink domain: G2S, reduction, and S2G writeback



Probability Transport

8 ranks; $H = 512$; $K = 36$; $E_{\text{local}} = 32$ ($E = 256$ total)

```
if constexpr (FORWARD_DISPATCH) {
  auto remote = out_prob[r] + idx * E * R + r * E;
  auto local  = &smem_prob[stage][r * E];
  cp_async_bulk(remote, local, E * sizeof(float));
}
if constexpr (BACKWARD_COMBINE) {
  auto remote = in_prob[s] + idx * E * R + s * E;
  cp_async_bulk(local, remote, E * 4, barrier);
}
```

Payload scale

Token: $H = 512$ BF16 $= 1024$ B. Full probability row: 32×8 FP32 $= 1024$ B/destination—the same as the token. At EP72: 9216 B ($9\times$ token).

Slice instead of row

Destination slice: 32 FP32 $= 128$ B. $8\times$ **less probability traffic on NVL8**; up to $72\times$ **at EP72**.

Measured dispatch gain

8-B300 selected-probability dispatch: $\sim$**200 GB/s** $\rightarrow$ $\sim$**600 GB/s** ($\sim 3\times$). Selected scalars remain for weighting and gradients; only rank-irrelevant slices are omitted.

Ballot Permutation

Standalone unfused path; B200 NVL8; $H = 512$; $T = 8192$; $E_{\text{local}} = 32$; $K = 36$

```
// permute_kernel<32>, DeepEP commit 1c3989c
int my_dest = token_routing[lane_in_warp];
unsigned mask = __ballot_sync(0xffffffff, my_dest > 0);

while (mask) {
  int expert = __ffs(mask) - 1;
  mask &= mask - 1;
  int dest = __shfl_sync(0xffffffff, my_dest, expert);
  for (int j = group_lane; j < hidden_size_fp4; j += 32)
    permuted_tokens_fp4[(dest - 1) * hidden_size_fp4 + j] =
        tokens_fp4[token_id * hidden_size_fp4 + j];
}
```

489 GB/s

permute

117 → 489

payload-model GB/s

351 GB/s

unpermute

173 → 351

payload-model GB/s

What the ballot removes

The reference assigns 128 threads/token and tests all 32 local-expert slots. Only about 4.5 are active, so 86% of slot checks, route loads, and branches are unnecessary.

Later upstream adoption. Tong Liu's #625 independently used an almost identical warp-ballot selected-route traversal; it was adopted after this path.

Scope. At $H = 512$, standalone unfused permute/unpermute outperforms fused. `unpermute_kernel<32>` (7ec3238) uses the same gather/accumulate traversal.

Combine Tuning

Stage shape and shared-memory budget; B200 NVL8, $H = 512$, $E_{\text{local}} = 32$, $R = 8$

G2S and S2G stage shapes

G2S holds one BF16 expert-output token, the source rank, a selected-probability slice, and producer/consumer state:

$$B_{\text{G2S}} = 512 \cdot 2 + 32 \cdot 4 + 2 \cdot 8 + 4 + 1 \simeq 1.17 \text{ KB}.$$

S2G holds one BF16 reduced token and the complete local probability row for backward:

$$B_{\text{S2G}} = 512 \cdot 2 + (32 \cdot 8) \cdot 4 = 2.00 \text{ KB}.$$

Capacity for in-flight work

The tuned $64/8$ allocation uses

$$64 \cdot 1.17 \text{ KB} + 8 \cdot 2.00 \text{ KB} \simeq 91.5 \text{ KB},$$

below the approximately 227 KB opt-in shared-memory limit; the default $10/2$ uses about 15.8 KB. This capacity keeps 64 G2S and 8 S2G entries in flight.

Combine Tuning

Default and tuned B200 settings

Single-NVL-domain configuration

Environment variable	Default	Tuned
NUM_SMS_COMBINE	24	32
NUM_OF_STAGES_G2S_COMBINE_API	10	64
NUM_OF_STAGES_S2G_COMBINE_API	2	8
NUM_OF_TOKENS_PER_GROUP_COMBINE_API	4	2

Per-pipeline allocation

Two pipelines divide the rings into 32 G2S and 4 S2G stages per pipeline. Both stage counts must be divisible by two.

Selection

Group 2 keeps two pipelines active while retaining sufficient G2S lookahead.

Combine Tuning

Group allocation and kernel throughput

Group-size effect

Group	G2S/pipe	S2G/pipe	Throughput
1 / 64 SMs	64	8	265 GB/s
2 / 32 SMs	32	4	263 GB/s
4 / 32 SMs	16	2	133 GB/s

Group 1 leaves a pipeline underused. Group 2 uses both pipelines and retains enough G2S lookahead for the same throughput with half as many SMs. Group 4 halves the FIFO again and no longer hides remote-read latency.

Combine kernel result

69 GB/s → 265 GB/s

default tuned, selected probabilities

The tuned standalone combine kernel improves output throughput by $3.8\times$. It uses a group of two and 32 combine SMs. The EP72 workload-specific tuple uses G2S 72, S2G 8, group 2, and 32 SMs.

Dispatch vs. Combine

NCU kernel profile: push versus pull, reduction, and synchronization

Dispatch / push

650 GB/s

72% of 900 GB/s NVLink peak
58% long-scoreboard stall
0.05% barrier stall

Combine / pull + reduce

271 GB/s

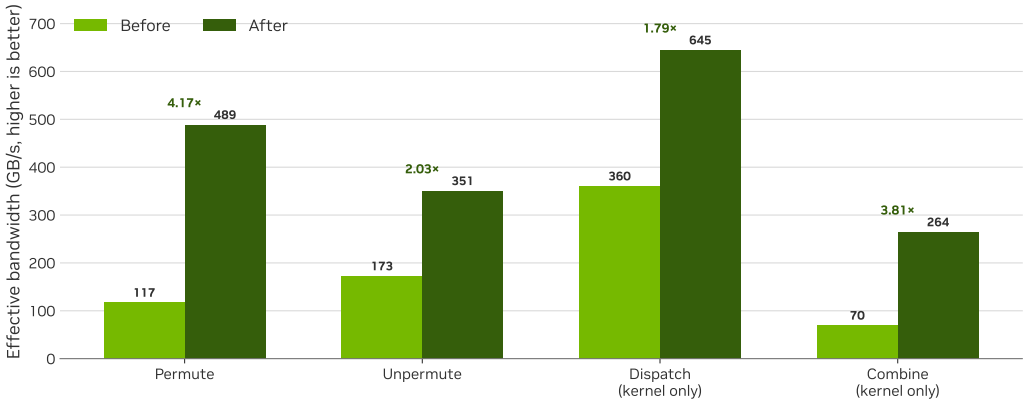
30% of 900 GB/s NVLink peak
27% long-scoreboard + 26% wait
15% barrier stall

Findings from NCU

Dispatch issues S2G writes and continues; its long-scoreboard stall is TMA/NVLink backpressure, with almost no inter-warp synchronization. Combine must wait for G2S request/response traffic, synchronize reduction warps, and store the accumulated output. The resulting wait and barrier stalls explain the $2.4\times$ throughput gap; fusing unpermute then serializes consumers behind slowly produced chunks and is about 14% slower than the standalone path.

Microbenchmarks

B200 NVL8, H=512, T=8192/rank, local-E=32, top-$k = 36$; matched kernel-only throughput



Dense Routing Map Scan

Selected expert IDs directly form rank and local-expert metadata

```
const int16_t* ids_in = input_topk_idx + token_id * TOPK;
const copy_t* src = reinterpret_cast<const copy_t*>(ids_in);
int16_t topk_ids[TOPK];

#pragma unroll
for (int j = 0; j < ROUTING_MAP_LOAD_ITER; ++j)
  *(reinterpret_cast<copy_t*>(topk_ids) + j) = src[j];

#pragma unroll
for (int k = 0; k < TOPK; ++k) {
  int expert = static_cast<int>(topk_ids[k]);
  if (node_begin <= expert && expert < node_end) {
    int rank = (expert - node_begin) / E_local;
    rank_mask[rank / 32] |= 1u << (rank % 32);
  }
}
```

32×

logical routing-metadata reduction: bool $[T, E]$ → int16 $[T, K]$

Input

Each `topk_indices` row holds K selected int16 global expert IDs. The scan vector-loads that row; -1 marks a dropped ID.

Metadata emitted

Each ID maps to a node and rank. The scan sets rank bitmasks and derives local-expert masks and per-expert token counts for dispatch.

Custom All-Gather

TMA pipeline writes route metadata to registered peer buffers

```
// elected warp leader, one double-buffered SMEM tile
mbarrier_arrive_expect_tx(mbars[slot], tma_size);
cp_async_bulk(shared_tile[slot], src + tile_offset,
              tma_size, mbars[slot]);

while (!mbarrier_try_wait_parity(mbars[slot], parity[slot])) {}
parity[slot] ^= 1;

for (int r = 0; r < rank_num; ++r)
  cp_async_bulk(dst_buffers_all_ranks[r] + dst_offset,
                shared_tile[slot], tma_size);
cp_async_bulk_commit_group();
```

Per-warp dataflow

Four warps own independent pipelines. Each double-buffers a 16 KB tile in shared memory, then issues TMA stores directly to IPC-mapped peer buffers.

Scope

Applies only within one NVLink domain with 16-byte-aligned metadata. Multi-node paths use NCCL.

Design Rejections

Profiling narrows the feasible design space

Direct permute

- Eliminates the staging buffer, but writes a token once for every routed expert.
- At $K = 8$, NCU observed $4.0\times$ more user NVLink TX and $2.2\times$ slower dispatch.
- At the target $H = 512$, $K = 36$, scattered writes and address work outweigh the standalone permute saving.

Fused combine

- Unpermute consumers poll until combine produces output chunks.
- Pull-based G2S reads and reductions make combine the slow producer.
- Best tuned path: $434\ \mu$s versus $381\ \mu$s standalone (14% slower).

Warp-pruned scan

- Builds a warp-union local-expert mask, then visits only its set bits.
- `__reduce_or`, `__ffs` control flow, and row initialization add coordination work.
- Results regressed on key templates; the simpler rank-bitset scan was retained.

MCore Integration

Contents

MCore Integration

Megatron Core

Integration boundaries

Collaborating workstreams

- Paged stash for bounded activation storage.
- Full-iteration CUDA graph capture and replay.
- 1F1B MoE communication/compute overlap.
- Capture-safe memory release without deferred-free inflation.

Delivered plumbing

```
if te.has_dense_route_output() and \
   dispatcher.accepts_dense_route():
    route = int16[T, K]
    fused_router(..., route_out=route)
    hybrid_ep(..., dense_route=route)
else:
    compatibility_bool_route()
```

Capability detection preserves backward compatibility.

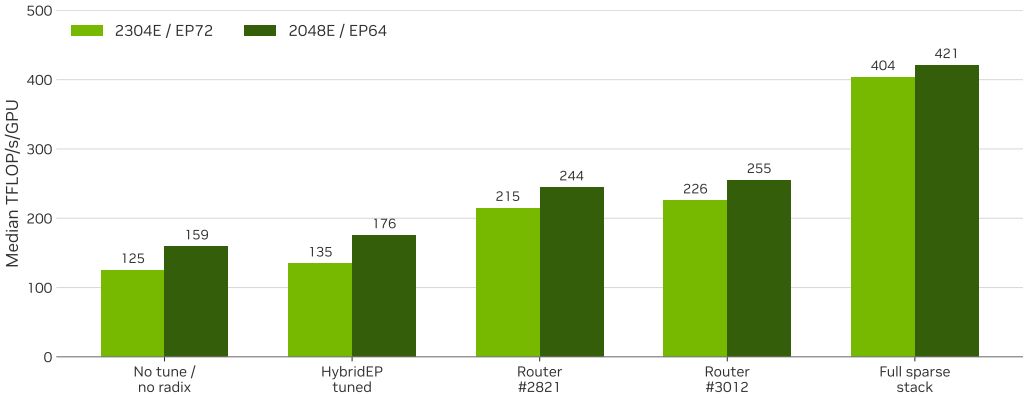
End-to-End Results

Contents

End-to-End Results

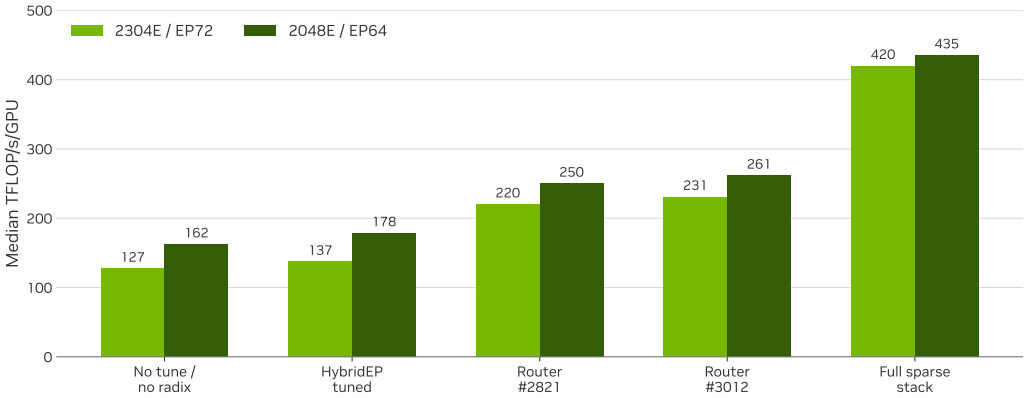
Full-Model Stages

GB200 NVL72; 1F1B, paged stash, full-iteration graph; median TFLOP/s/GPU



Full-Model Stages

GB300 NVL72; 1F1B, paged stash, full-iteration graph; median TFLOP/s/GPU



Benchmark Shapes

MoE dimensions in the matched MoE-only suite

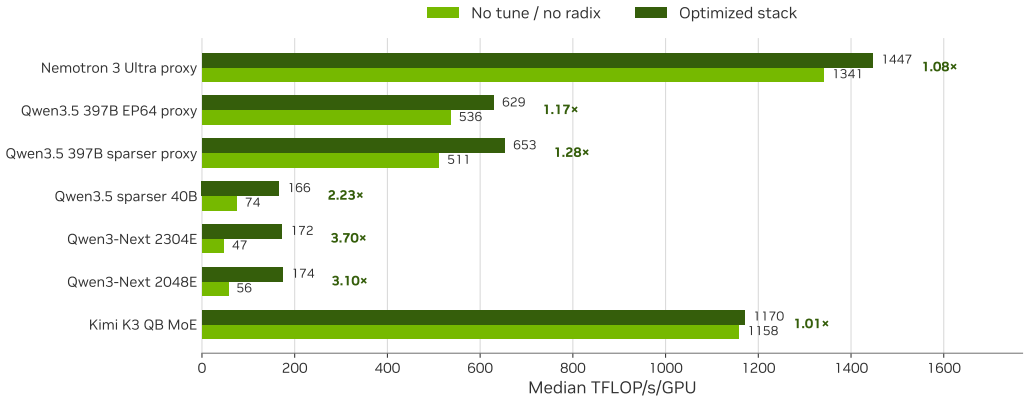
Common protocol: mock data, MoE-only frontend, full-iteration CUDA graph, and paged stash. H_{hid} is the routed-expert intermediate width; an em dash denotes no latent projection.

Model	H	H_{latent}	H_{hid}	Shared expert(s)	E	K
Nemotron 3 Ultra proxy	8192	2048	5120	1×10240	512	22
Qwen3.5 397B EP64 proxy	4096	–	1024	1×1024	512	10
Qwen3.5 397B sparser proxy	4096	–	1024	1×1024	1024	16
Qwen3.5 sparser 40B	2048	512	512	1×512	1152	18
Qwen3-Next 2304E	2048	512	512	1×512	2304	36
Qwen3-Next 2048E	2048	512	512	1×512	2048	32
Kimi K3 QB MoE[1]	7168	3584	3072	2×3072	896	16

[1] Standalone 12-layer QB MoE mock: K3's SiTU-GLU activation, quantile balancing, and latent up-projection normalization are implemented; MCore does not provide the other K3 components.

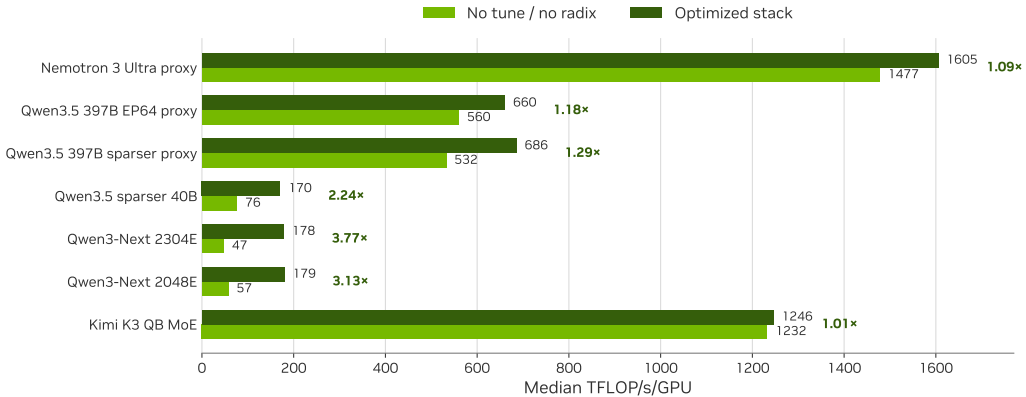
MoE-Only Benchmarking

GB200 NVL72; matched full graph + paged stash; median TFLOP/s/GPU



MoE-Only Benchmarking

GB300 NVL72; matched full graph + paged stash; median TFLOP/s/GPU



MoE-Only Takeaways

Generalization across wide, sparse expert shapes

3.77×
Qwen3-Next 2304E
47.2 → 177.9

3.13×
Qwen3-Next 2048E
57.4 → 179.4

2.24×
Qwen3.5 sparser 40B
76.1 → 170.3

Limiting cases

Nemotron's proxy shape improved only 1.09×: when expert compute is already large, router and sparse communication are a smaller fraction of the module. Kimi K3 QB MoE was also marginal: 1158 → 1170 TFLOP/s/GPU on GB200 and 1232 → 1246 on GB300 (about 1.01×). This is MoE-only, without 1F1B; full graph and paged stash are matched.

Conclusions

Key findings and applicability limits

Findings

- Radix selection resolves the pathological large-E, large-K case.
- Compact int16 routes reduce logical metadata 32$\times$.
- Ballot skips inactive local experts.
- Combine performance requires coordinated queue, group, and batch tuning.
- The stack is most effective when E and/or K are large and routed-expert H_{hid} is sufficiently small.

Scope

- Do not aggregate isolated speedups into end-to-end predictions.
- The EP72 tuple is not a universal default.
- Training retains selected probability transport.
- Separate this work from graph, stash, and 1F1B contributions.
- Distinguish MoE-only and full-model measurements.

Thank You

Questions and discussion are welcome.